21 · 全链路功能测试报告与完成度评估

项	内容
文档编号	DT-PdM-21
测试对象	谛听 PdM · 诊断服务 v0.2.1（集群内 2 副本）
数据规模	1 亿条特征点（TiDB）+ 2400 台设备快照 + 1201 条 Redis L2 缓存
结论	E2E 23/23 通过；完成度 Ph1 核心 100%，距"完整产品"还差 5 块
日期	2026-10-21

---

1. 你问的三个问题，直答

Q1：项目完整完成了吗？

没有。 完成度分区：

模块	状态	说明
信号处理核（pdm_signal）	100%	44 项单测，FFT/包络/轴承/HI 全实现
诊断服务（Litestar+Granian）	100%	已部署集群，E2E 23/23
数据层（TiDB 1 亿行 + 多租户）	100%	18 张表、租户守卫、快照
缓存（L1+L2+warmup）	100%	warmup 1201 台/20s，L2 跨重启命中已验证
决策会诊（L4）	0%	方案卡/四角色/Evaluator 只有文档，无代码
MCP 工具层	0%	接口契约有，服务未实现
Rust 采集/DSP	0%	边缘侧未启动
Go 事件管线/BFF	0%	未启动
Java 资产/知识/集成	0%	未启动
Pulsar 事件	阻塞	等 tenant 权限（I1）
备份（BR）	未做	1 亿行数据无备份，最紧急

Q2：可以进行全端全功能全链路测试吗？

已完成的范围：可以，且刚刚跑完——23/23 通过。 "全端"还做不到：Rust 边缘、Go 管线、Java 集成、决策会诊都还没实现，链路到"诊断服务"为止。

Q3：造数完成了吗？Redis 有缓存并支持 warmup 吗？

全部完成：

项	数值
feature_point	100,000,000 行（精确）
资产 / 测点	2,400 台 / 20,400 个
四级分布	绿 77,870,435 / 黄 20,610,197 / 橙 1,517,847 / 红 1,521

项	数值
时间跨度	2026-03-26 → 2026-10-21（209 天，1 小时粒度）
生成速度	6 进程 × ~8,600 行/s = 51,000 行/s，33 分钟完成
asset_metric_snapshot	2,400 行（每台设备 5 维完整快照）
Redis L2	1,201 键（`pdm:diag:*`，db 3，TTL 120s）
warmup	`GET /warmup?limit=N`，1201 台 / 20.1 秒，幂等

---

2. E2E 测试结果（23/23）

组	用例	结果
T1 基础	healthz / version / model	3/3
T2 数据	1 亿行 / 2400 设备 / 四级分布 / 快照非空	4/4
T3 功能	诊断 200 / 不降级 / 5 维 / 证据 / 权重归一 / 免责 / 等级一致	7/7
T4 缓存	结果可用 / 二次命中 / 一致 / L2 启用 / Redis 键	5/5
T5 性能	端到端样本（服务端 elapsed_ms 14 – 17ms）	1/1
T6 安全	租户 403 / 缺 header 400 / 无指标 422	3/3

服务端真实耗时（集群内直测，排除网络）：

```
■■■L1 ■■■■L2 ■■■: elapsed_ms = 17
■■■■:                 elapsed_ms = 14-16
```

我测试机到集群的网络 RTT 为 p50 586ms（VPN 链路），与服务无关。

2.1 后续演进：E2E 已扩至 66 项 + 浏览器层 46 项

本节记录的是早期基线（23 项）。此后 E2E 持续扩展，当前状态：

层	覆盖	结果
E2E（httpx，集群内 API）	Pulsar 消息、定时备份/清理、方案卡多角色确认、趋势 API、EAM 投递重试、看板鉴权	66/66
无头浏览器（Playwright + Chromium）	看板渲染、诊断/趋势/会诊三个交互路径、JS 异常与资源加载	46/46
单元测试	规则引擎、仓储、事件	72 项

浏览器层的价值在 2026-10-23 得到验证——它抓出了 API 层完全测不到的 3 个缺陷：

- 看板「诊断」主按钮绑 `diagnose()`、实际函数 `diag()` → 点击报 ReferenceError
- 线上看板是旧版（趋势图/诊断分解在仓库里但 ConfigMap 未更新）
- java-asset 详情接口表名 `measure_point` 应为 `measurement_point` → 恒 500

共同点：只有人真的去点、真的去看才会暴露。httpx 不执行 JS、不渲染 DOM，
也发现不了"部署的静态文件比仓库旧"。

> 完整记录见 31_无头浏览器全端全链路测试报告。 结论已回写至 11_验收标准与上线检查单 第 3.9 节与 T-24h 清单。

---

3. 本轮解决的"服务器无外网"问题（完整结论）

3.1 诊断过程（三层澄清）

测试	结果	结论
`curl https://charts.pingcap.org`	HTTP 000，0.06s	立即拒绝
`ping mirrors.aliyun.com`	26ms，0 丢包	网络层通
`curl -4 https://mirrors.aliyun.com`	200，TLS 0.32s	HTTPS 通
下载 39KB 文件	稳定 128 KB/s	出口限速
`pip install pymysql`（容器内）	成功	pip 完全可用

真相：服务器有外网，但出口限速约 128 KB/s，且 IPv6 完全不通。 之前所有"失败"都源于：大包下载超时 + IPv6 优先导致的 connect 挂起 + curl 无 `-4`。

3.2 彻底解决方案（已落地）

策略	实施
小依赖在线装	`pip -i https://mirrors.aliyun.com/pypi/simple/ --timeout 120 --retries 10`，容器内直装成功
大包离线 wheel	本机 `pip download --platform manylinux2014_x86_64` → 60MB 传上去 → `pip --no-index --find-links`
CPU 兼容	服务器 Xeon 5320 不支持 AVX2，numpy 2.4.6（X86_V2 基线）无法 import → 钉死 numpy==1.26.4（SSE 基线）
依赖整树解析	放弃逐个猜传递依赖（multidict/click/litestar-htmx...），改为容器内 pip install 全树 → docker commit 成 deps 层
分层镜像	`:deps`（依赖层，已固化+验证）→ `:0.2.x`（应用层，秒级重建）

踩坑记录：

- musllinux wheel（alpine 下载的）装进 glibc（python:3.11-slim）失败 → 必须用 `--platform manylinux2014` 交叉下载
- `uvicorn`/`pytest<9`/`redis<6` 等版本上限与新版本冲突 → 移除上限或直接删依赖
- `pip download` 不带 `--platform` 时在 alpine 里只会拉 musllinux 版

3.3 对 TiKV/PG 议题的修正

外网实际可用后，TiDB Operator 理论上可装（需 ~2GB 镜像 × 128KB/s ≈ 4.5 小时，可行）。是否迁移重新变成开放决策，留给你：

选项	成本	收益
维持手工 StatefulSet	0	已验证可用，缺自动编排
迁 TiDB Operator	~5h 下载 + 迁移	自动扩缩容/升级/备份集成

---

4. 关键基础设施状态

```
TiDB:  PD 3/3(SS) · TiKV 3/3(SS,200Gi/■■) · TiDB 2/2 · TiFlash 1/1 ── ■ Running
■■:  7 PV ■ Bound■760Gi of 3T■■Retain ■■
■■:  pdm-prod/python-diagnosis ×2 ■■■:0.2.1■■HPA 2–6
■■:  http://10.167.2.175:32231/pdm-api/* ■Host: pdm-api.10.167.2.175.nip.io■
       ■ ■■■ /pdm-api ■■■dify-catchall■■ host■/ Prefix■■■■■■■■■
■■:  L1 ■■■(30s) + L2 Redis db3(120s) + /warmup ■■■■
```

Ingress 踩坑记录：dify 的 catch-all ingress 没配 host、占用了 `/` Prefix，会与任何无更长前缀的规则竞争。解法是 PdM 用专属长前缀 `/pdm-api` + regex rewrite `/$2`。另发现该集群 ingress 的 NodePort 对 keep-alive 连接有路由不一致（首请求 200 后续 404），测试端已用 `max_keepalive_connections=0` 规避，生产接入需改用独立域名+独立 Ingress 或直连 NodePort。

---

5. 偏差登记（有意为之，需回正）

#	偏差	原因	回正条件
D1	特征点放 TiDB 而非 TDengine	本地仓库无 TDengine 镜像；1 亿行验证 TiDB 也够	TDengine 镜像同步后迁移
D2	诊断落库用 `elapsed_ms` 同步写	简化；量小可接受	Ph3 改为 Outbox 异步
D3	`pytest-asyncio` 从 requirements 移除	离线 wheel 缺失；单测不依赖 async	有网后恢复
D4	未用 TiDB Operator	下载 4.5h；手工 SS 已验证	你拍板是否迁

---

6. 下一步优先级（建议顺序）

- BR 备份（1 亿行数据现在无备份，最高优先）
- 决策会诊服务（L4，价值最高的缺失模块）
- MCP 服务（让 LLM 能调用诊断能力）
- Rust 边缘采集（接真实设备）
- Go 事件管线（接 Pulsar）
- TiDB Operator 迁移决策（可选）